SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:		**Reg. No.:**	
Date:		**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
    - Carefully understand the given problem statement.
    - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements**
    - List all functional requirements required by the proposed system.
    - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
    - Identify all primary and secondary actors interacting with the system.
    - List the major use cases associated with each actor.
    - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
    - Introduction
    - Problem Description
    - Scope of the System
    - Functional Requirements
    - Non-Functional Requirements
    - Use Case Descriptions
    - Data Requirements
    - External Interface Requirements
    - System Features
5. **Identify Assumptions and Constraints**
    - List the assumptions considered while developing the system.
    - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
    - Verify whether all stakeholder requirements have been addressed.
    - Check for ambiguity, redundancy, inconsistency, and missing requirements.
    - Suggest improvements to enhance the quality and completeness of the requirement specification.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.

Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS) Documentation (15%)	SRS is well-structured, complete, professionally organized, and includes all required sections.	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25

HONEY CHAIN: A Blockchain-Based System for Honey Traceability and Smart Beekeeping Management

Software Requirements Specification (SRS)

1. Study and Analyze the Scenario

1.1 Business Domain

The proposed system belongs to the agriculture, beekeeping, food supply chain, and blockchain technology domains. It aims to improve honey quality, transparency, traceability, and beekeeping management.

1.2 Stakeholders

- Beekeepers: Manage bee colonies, record honey production, and monitor hive conditions.
- Honey Producers: Process, package, and maintain honey quality records.
- Distributors: Track honey batches during transportation.
- Retailers: Verify product origin and authenticity.
- Consumers: Scan a QR code to view honey history and source details.
- Administrators: Manage users, permissions, and system operations.
- Quality Inspectors: Verify honey quality and certification records.

1.3 Existing Challenges

- Difficulty verifying the origin and authenticity of honey.
- Lack of transparency in the supply chain.
- Risk of fake or adulterated honey products.
- Manual recordkeeping and inefficient hive monitoring.
- Difficulty tracking contaminated or defective honey batches.
- Limited access to reliable production and quality information.

1.4 Objectives

- Provide transparent, tamper-evident honey traceability using blockchain.
- Monitor beehive conditions using IoT sensors, where available.
- Maintain production, processing, and distribution records.
- Generate QR codes for honey batches.
- Enable consumers to verify product information.
- Improve quality control and beekeeping efficiency.

2. Functional and Non-Functional Requirements

2.1 Functional Requirements

ID	Requirement
FR1	The system shall allow users to register and log in securely.
FR2	The system shall support role-based access control.
FR3	Beekeepers shall register apiaries, hives, and bee colonies.
FR4	The system shall record honey harvesting details and batch IDs.
FR5	The system shall record processing, packaging, and quality test results.
FR6	The system shall store traceability events on a blockchain or record their cryptographic hashes on-chain.
FR7	The system shall generate a unique QR code for each honey batch.
FR8	Consumers shall scan QR codes to view the product's traceability history.
FR9	Distributors and retailers shall update shipment and receipt information.
FR10	The system shall collect hive temperature, humidity, and other available sensor readings.
FR11	The system shall generate alerts when monitored hive conditions exceed configured limits.
FR12	Administrators shall manage users, records, and permissions.
FR13	The system shall generate production, quality, and traceability reports.
FR14	The system shall maintain audit logs of important actions.
FR15	The system shall flag missing, invalid, or inconsistent batch records.

2.2 Non-Functional Requirements

Requirement	Description
Performance	Common pages should load within 3 seconds under normal conditions.
Security	Use encrypted communication, secure authentication, and role-based access.
Reliability	Prevent loss or corruption of critical records through backups and validation.
Usability	Provide a simple, mobile-friendly interface.

Scalability	Support increasing numbers of users, hives, and honey batches.
Maintainability	Use modular software components and documented APIs.
Availability	Target 99.5% monthly uptime, excluding planned maintenance.
Integrity	Detect unauthorized changes to recorded traceability events.
Privacy	Restrict access to personal and commercially sensitive information.
Interoperability	Support standard APIs for sensors and external systems where available.

These performance and availability values are proposed acceptance targets and should be confirmed with stakeholders.

3. Identify Actors and Use Cases

3.1 Actors

Actor	Role
Beekeeper	Registers hives and records harvesting details.
Honey Producer	Records processing, packaging, and quality details.
Distributor	Updates transportation and delivery records.
Retailer	Confirms receipt and sale availability.
Consumer	Scans QR codes and views product history.
Quality Inspector	Verifies laboratory results and certifications.
Administrator	Manages accounts, permissions, and system settings.
IoT Sensors	Send hive condition measurements to the system.
Blockchain Network	Validates and stores traceability transactions.

3.2 Major Use Cases

- Register, log in, and manage user profiles.
- Register apiaries and monitor beehives.
- Record honey harvesting and create batches.
- Update processing and packaging information.
- Upload quality test results and certifications.
- Record blockchain transactions.
- Generate and scan QR codes.
- Track shipment and delivery status.
- Receive hive condition alerts.

- View reports and audit logs.

4. Software Requirement Specification (SRS)

4.1 Introduction

HONEY CHAIN is a blockchain-based system designed to improve honey traceability and smart beekeeping management. It combines digital records, blockchain verification, QR codes, and optional IoT sensors to maintain transparent information from honey production to the consumer.

4.2 Problem Description

Traditional honey supply chains often depend on paper records or disconnected databases. This makes it difficult to verify honey origin, identify adulteration risks, track shipments, and monitor hive conditions. HONEY CHAIN addresses these issues by maintaining traceability records and providing a convenient way to verify product information.

4.3 Scope of the System

The system covers:

- Beekeeper and hive management.
- Honey harvesting and batch registration.
- Processing, packaging, and quality documentation.
- Blockchain-supported traceability.
- QR-based product verification.
- Distribution and retail tracking.
- Optional IoT-based hive monitoring.
- Alerts, reports, and audit records.

The system does not independently prove that honey is pure merely because a record exists on the blockchain. Physical authenticity still requires trustworthy inspections, laboratory testing, and verified data entry.

4.4 Functional Requirements

The system shall provide the following main features:

- User management: Register, authenticate, and authorize users.
- Hive management: Store hive IDs, locations, and condition readings.
- Harvest management: Record harvest dates, quantities, and batch identifiers

- Quality management: Maintain test results, certificates, and inspection details.
- Blockchain traceability: Record or verify the integrity of important supply chain events.
- QR code management: Link each batch to its traceability record.
- Supply chain tracking: Record shipment, receipt, and transfer events.
- IoT monitoring: Collect sensor readings and issue configurable alerts.
- Reporting: Generate production, quality, shipment, and audit reports.
- Administration: Manage permissions, system configurations, and invalid records.

4.5 Non-Functional Requirements

The system must be secure, reliable, scalable, maintainable, accessible, and easy to use. It should validate all incoming data, support recovery from failures, and preserve the integrity of traceability records. Sensor outages and blockchain transaction failures must be handled without silently losing data.

4.6 Use Case Descriptions

Use Case 1: Register Honey Batch

Field	Description
Primary actor	Beekeeper
Precondition	The beekeeper is authenticated.
Main flow	Enter harvest date, quantity, hive ID, and location; validate data; assign a batch ID; save the record; register its traceability event.
Alternative flow	If data is invalid, display an error and request correction.
Postcondition	The batch is registered and available for authorized tracking.

Use Case 2: Verify Honey Origin

Field	Description
Primary actor	Consumer
Precondition	The honey batch has a valid QR code.
Main flow	Scan QR code; retrieve the batch history; verify record integrity; display origin, harvest, quality, and shipment information.
Alternative flow	If the code is invalid or records are unavailable, display an appropriate warning.
Postcondition	The consumer can view the available verified traceability information.

Use Case 3: Monitor Hive Conditions

Field	Description
Primary actor	Beekeeper
Supporting actor	IoT sensors
Precondition	Sensors are registered and configured.
Main flow	Collect readings; validate measurements; store readings; compare them with configured thresholds; send alerts when required.
Alternative flow	If a sensor fails or readings are missing, flag the monitoring interruption.
Postcondition	Hive conditions and any alerts are available for review.

Use Case 4: Update Shipment

Field	Description
Primary actor	Distributor
Precondition	The honey batch is registered and ready for shipment.
Main flow	Enter shipment ID, dispatch details, destination, and status; validate the information; save the event and update traceability history.
Alternative flow	Reject unauthorized or inconsistent updates.
Postcondition	The shipment status is updated and its event is auditable.

4.7 Data Requirements

Entity	Important attributes
User	User ID, name, role, contact details
Apiary	Apiary ID, location, beekeeper ID
Hive	Hive ID, apiary ID, sensor ID
Sensor Reading	Reading ID, hive ID, timestamp, temperature, humidity
Honey Batch	Batch ID, hive ID, harvest date, quantity, status
Quality Record	Test ID, batch ID, results, inspector, certificate reference
Processing Record	Record ID, batch ID, processing date, packaging details
Shipment	Shipment ID, batch ID, sender, receiver, status
Traceability Event	Event ID, batch ID, event type, timestamp, digital signature or hash
Audit Log	Log ID, user ID, action, timestamp

Personal details and large files should generally be stored in a protected database or file store. Blockchain can retain hashes and references to records rather than exposing sensitive information publicly.

4.8 External Interface Requirements

- User interface: Responsive web or mobile interface for beekeepers, producers, distributors, inspectors, and consumers.
- Hardware interface: Optional IoT temperature and humidity sensors.
- Software interface: Database, blockchain node or smart contract, QR code generator, and notification service.
- Communication interface: Secure HTTPS APIs for transferring data.
- Reporting interface: Downloadable reports and dashboards.
- QR interface: Mobile camera scanning that opens the batch verification page.

5. Identify Assumptions and Constraints

5.1 Assumptions

- All authorized stakeholders have access to a smartphone or computer.
- Users provide accurate harvest, processing, and shipment information.
- Blockchain services are available when transactions need to be recorded.
- IoT sensors are optional and available only at equipped apiaries.
- Each honey batch has a unique identification number and QR code.
- Quality inspectors provide valid laboratory results and certificates.
- Consumers can access the public product verification page through the internet.

5.2 Constraints

Constraint type	Description
Technical	Limited sensor connectivity, blockchain transaction costs, and integration complexity.
Operational	Beekeepers require training to enter records correctly.
Economic	Sensor devices, hosting, maintenance, and blockchain infrastructure incur costs.
Legal	The system must comply with applicable food safety, labeling, privacy, and data protection requirements.
Time	Development and testing must be completed within the assigned project schedule.
Environmental	Outdoor sensors must withstand rain, heat, dust, and other weather conditions.
Network	Poor internet connectivity may delay synchronization and alerts.

6. Validate Requirement Completeness and Consistency

6.1 Validation Checklist

Validation criterion	Result
Stakeholders identified	Covered
Functional requirements defined	Covered
Non-functional requirements defined	Covered
Actors and use cases identified	Covered
Data requirements specified	Covered
External interfaces described	Covered
Assumptions and constraints documented	Covered
Security and privacy considered	Covered
Blockchain and IoT failure handling	Partially covered; detailed recovery rules are needed
Testing and acceptance criteria	Require further definition

6.2 Ambiguity, Redundancy, and Inconsistency Checks

- Ambiguity: Define exactly which blockchain events are recorded and which users can access them.
- Redundancy: Avoid storing identical batch details separately in multiple modules without a shared batch ID.
- Inconsistency: Ensure harvest quantities, processing quantities, and shipment quantities remain logically consistent.
- Missing requirements: Specify backup procedures, transaction failure handling, data retention, and account recovery.
- Security gap: Verify the authenticity of users entering quality results and protect sensitive data.
- IoT gap: Define sensor calibration, missing readings, alert thresholds, and offline operation.

6.3 Suggested Improvements

- Establish measurable acceptance criteria for every major functional requirement.
- Use role-based permissions and digital signatures for important updates.
- Implement automated tests for QR verification, blockchain transactions, and invalid records.
- Maintain a complete audit trail for changes and failed operations.
- Use offline data collection with secure synchronization when connectivity returns.
- Conduct usability testing with beekeepers and consumers.
- Test system performance with increasing numbers of users, batches, and sensor readings.

Conclusion

HONEY CHAIN provides a proposed solution for improving honey traceability, quality documentation, and smart beekeeping management. By combining blockchain, QR codes, secure digital records, and optional IoT monitoring, the system can improve supply chain transparency and consumer confidence. Proper validation, access control, and reliable data collection are essential to ensure the system meets its objectives.